
[Re] Probabilistic Plan Recognition Using Off-the-Shelf Classical Planners

Okwukwechukwu Mbajiorgu, Zaid Ismail, Michael Anokye-Boateng
School of Computer Science and Applied Mathematics
University of the Witwatersrand
Johannesburg, South Africa

Reproducibility Summary

Scope of Reproducibility

We reproduce the experiments of Ramírez and Geffner [2010], who describe a goal recognition method that compiles each (problem, observation) pair into two classical planning problems and converts the resulting cost difference into a posterior over candidate goals. We test four claims drawn from Table 1 of the original paper. The first three concern the behaviour of the recognition quality Q and the spread $|G^*|$ across observation percentages and domains. The fourth is an extension claim about the role of the Boltzmann constant β , which the original paper does not specify.

Methodology

We re-implemented the pipeline from scratch in Python as four modules: a PDDL compiler, a Fast Downward subprocess wrapper, a Boltzmann scoring layer, and an evaluation driver. We did not use the authors' code. Benchmark domains come from the authors' distribution. To keep the loop uniform across domains, we sample observations from precomputed optimal plans in every domain, including Campus and Kitchen where the paper samples from stochastic suboptimal traces. This simplifies the pipeline at the cost of making those two domains structurally easier to recognise. The $\beta = 1.0$ run took roughly two hours for 1885 trials. The full β sweep across six values took close to sixteen hours.

Results

Recognition quality Q reproduces in every directly comparable cell. Because we sample from optimal plans, Q saturates at 1.00, making Claims 1 and 2 trivially true under Q . We therefore read both claims through the spread metric. Spread decreases monotonically in all six of our domains and in four of six in the paper, supporting Claim 1. It falls below 2 by 50% observation in five of six domains in both reproductions, supporting Claim 2 with Block-Words as the common failure mode. At low observation percentages our spread exceeds the paper's by factors of 1.6 to 2.3 on Block-Words, Intrusion-Detection and Kitchen. The β sweep shows that Q and $|G^*|$ are mathematically invariant under β for integer action costs. Only the posterior mass on the true goal $P(G^* | O)$ varies with β , from near-uniform at $\beta = 0.1$ to sharp at $\beta = 10$.

What was easy

The compilation in Definition 2 of the paper is mechanical once read carefully. We translated it to about a hundred lines of Python without difficulty. Fast Downward solved every compiled problem it was given. Four of the six domains shipped with ready-to-use PDDL files, which removed the need to generate observations from scratch in those domains.

What was difficult

The paper does not state which value of β was used, and we spent time investigating this before concluding that the rank-based metrics are invariant under β . Two of the shipped PDDL files contained syntax errors that Fast Downward rejected. The Campus and Kitchen domains shipped as Python 2 generator scripts that needed porting. The Grid domain ships only two instances, so we hand-expanded the set to get a usable sample size.

1 Introduction

The rapid growth of artificial intelligence and autonomous systems has increased the importance of accurately predicting and optimising the behaviour of intelligent agents. Goal recognition, the task of inferring an agent’s intended objective from observations of its behaviour [Elsevier nd], is central to this challenge, with applications spanning natural language understanding [Cohen *et al.* 1981], multi-agent coordination [Huber *et al.* 1994], intrusion detection [Geib and Goldman 2002], and assisted cognition Pentney *et al.* [2006]. Classical approaches to this problem have relied on pre-compiled plan libraries or policy sets [Kautz and Allen 1986; Geib and Goldman 2009], which constrain both flexibility and scalability.

A more principled class of approaches replaces the plan library with a domain theory from which plans are generated on demand. Planning-based methods simplify goal recognition by converting it into classical planning problems that can be solved by existing planners. Ramírez and Geffner [Ramírez and Geffner 2009] introduced this perspective, demonstrating that goal recognition can be solved by invoking a lightly modified planner. Their later work extends goal recognition from a binary decision problem to a probabilistic one by estimating the posterior distribution $P(G | O)$ for each candidate goal. Rather than producing a simple accept-or-reject judgement, the method evaluates the likelihood of a goal by comparing the cost of achieving it while complying with the observed actions against the cost of achieving it without complying with those observations. This inference is performed through only two calls to an unmodified classical planner.

This report presents an independent reproduction of the experimental results reported in [Ramírez and Geffner 2010]. We re-implement the PDDL compilation procedure described in Definition 2 and Proposition 3 of the paper, integrate Fast Downward [Helmert 2006] as the underlying planner, and evaluate the method across the planning benchmark domains considered in the original work. Our aim is to assess whether the quantitative claims of the paper, specifically the recognition quality metric Q and goal spread S reported in Table 1, can be independently verified, and to document any ambiguities, implementation choices, or discrepancies encountered along the way.

The remainder of the report is organised as follows. Section 2 lists the four claims we test. Section 5 reports our experimental results and indicates which claims are supported by our data. The final discussion section comments on what was easy and what was difficult to reproduce.

2 Scope of reproducibility

This work reproduces the experiments of [Ramírez and Geffner 2010], who propose a probabilistic goal recognition method that uses existing classical planners without modifying their internal algorithms. Given a planning domain, a set of candidate goals, and a partially observed action sequence, the method computes a posterior probability distribution $P(G | O)$ over candidate goals by invoking an unmodified planner twice per goal. The first call finds the cheapest plan that complies with the observations. The second call finds the cheapest plan that does not. The resulting cost difference drives a Boltzmann likelihood which, combined with a uniform prior via Bayes’ rule, gives the posterior distribution. The original paper tests the method across six domains with both optimal and non-optimal planners, reporting recognition quality Q (the fraction of trials in which the true goal is among the most likely) and spread S (the average number of tied top-ranked goals).

We test the following claims derived from the experimental results reported in Table 1 of the original paper:

- **Claim 1:** Under optimal planning, the recognition quality Q is non-decreasing as the percentage of observed actions increases across all six planning domains.
- **Claim 2:** Under optimal planning, the spread $|G^*|$ is at most 2 by 50% observation in all six domains. We adopt this threshold as an operationalisation of the paper’s qualitative claim that the method shows “good precision”. The paper itself reports the spread values in Table 1 but does not single out a numerical threshold.
- **Claim 3:** Under optimal planning, the average spread S converges to 1 at 100% observation across all six domains, indicating that complete observations yield an unambiguous prediction.
- **Claim 4 (Extension):** The effects of the parameter β in the Boltzmann likelihood, which is unspecified in the original paper.

3 Background

Planning is an agent’s ability to carry out a sequence of actions from its initial state to achieve a goal state. While a domain defines an area in which a set of goals can be achieved by the agent. Planning recognition involves observing

an agent’s actions within a domain and then determining the goal the agent is working towards from the sequence of actions it performs. However, in a domain, there exist multiple possible goals that can be true. Since there may be multiple possible goal states within a domain, and an agent may not act rationally or follow the optimal action sequence from its initial state to its goal state, it is not immediately clear which goal an agent is targeting along its way.

This is why probabilistic plan recognition is introduced to form a probability distribution over the goals an agent could be targeting within a domain, given the set of observations of its actions at a given time. These probabilities are determined using Bayes’ rule. The probability distribution ultimately shows the most likely goal an agent is trying to achieve given an observation of its actions, i.e., the posterior. To find this probability, a prior probability distribution over all the goals is already known, but the likelihood element of Bayes rule, which specifies the probability of an observation sequence given a goal, is determined by computing the difference between the cheapest cost of compliant plans to obtain a goal and the cheapest cost of non-compliant plans that lead to the agent still reaching its goal state.

When handling observations of an agent’s actions with off-the-shelf planners to compute costs, the original planning domain must be transformed into a new domain to identify the compliant and non-compliant plan costs. In this new domain, a fluent is evaluated for each observed action and linked together. This means a fluent is true only if it satisfies the observations at each step up to its current point. This makes it possible to compute a planning problem to find the cheapest compliant plan and the cheapest noncompliant plan for each goal state. These can then be used to calculate the likelihoods needed to get the posterior probability.

The likelihood is calculated assuming a Boltzmann distribution, and the difference between the two costs influences the posterior probability. The lower the difference, the higher the probability, and the opposite when the difference is higher. This implies that it is easier to predict the observations for smaller differences. With this likelihood and prior distribution, it is then possible to compute the probability distribution over all goals in the domain.

The method proposed in the paper by .Ramírez and Geffner [2010] addresses the problem of goal recognition for an agent in a domain. The method first transforms the original domain into a new domain. Then, for each candidate goal, the costs of the cheapest compliant and non-compliant plans are computed using a classical planner. The difference between these costs is used to calculate the likelihood of the observations given each goal, which is then combined with a prior distribution over goals to produce a posterior probability distribution over all possible goals. This probability distribution is updated as new observations are made.

4 Methodology

We re-implemented the recognition pipeline from scratch in Python based on Definitions 2 and 3 and the Boltzmann likelihood given in [Ramírez and Geffner 2010]. The classical planner is Fast Downward [Helmert 2006] invoked as a subprocess. Benchmark domains were obtained from the authors’ distribution at <https://sites.google.com/site/prasplanning>. We did not use the authors’ code.

4.1 Model descriptions

The recognition method has no learned parameters. It is an algorithmic pipeline with the following four stages, each implemented as a single Python module:

1. **PDDL compiler** (`src/prp/compiler.py`). For each candidate goal G and observation sequence O , produces two compiled PDDL pairs. The first contains plans that must include the observed actions ($G + O$). The second contains plans that must omit at least one of them ($G + \overline{O}$). The transformation introduces a chain of fresh fluents $obs_0, \dots, obs_m$ and duplicates each observed action schema. Each duplicate binds its parameters to the observed arguments and chains the obs_i fluents through preconditions and effects.
2. **Planner wrapper** (`src/prp/planner.py`). Invokes Fast Downward as a subprocess, parses the resulting `sas_plan` or pre-computed `.soln` file, and exposes a `plan(domain, problem) → (actions, cost)` interface.
3. **Scoring** (`src/prp/scoring.py`). Converts the vector of cost differences $\Delta(G_k, O) = c(G_k + O) - c(G_k + \overline{O})$ for the K candidates into a posterior using a Boltzmann likelihood and uniform prior: $P(G_k | O) \propto \exp(-\beta \cdot \Delta(G_k, O))$. Numerical stability is handled by subtracting $\max_k(-\beta \cdot \Delta_k)$ before exponentiation.
4. **Evaluation driver** (`src/prp/evaluate.py`). Iterates over (domain, instance, true goal, observation %) combinations, calls the compiler and planner, computes the posterior, and writes one CSV row per trial. The driver is resumable. Interrupted runs continue from the last completed row.

4.2 Environments

We use the six benchmark domains reported in the original paper. Table 1 summarises the size of each domain and the number of recognition trials per cell in our experiment.

Table 1: Benchmark domains. $|\mathcal{G}|$ is the average number of candidate goals per instance. “Instances” counts the number of distinct (objects, initial state) tuples used. “Trials/cell” is the number of recognition trials averaged at each observation percentage.

Domain	$ \mathcal{G} $	Instances	Trials/cell
Block-Words	20.0	2	40
Campus	2.0	50	100
Grid	2.6	8	21
Intrusion-Detection	15.0	2	30
Kitchen	3.0	50	150
Logistics	7.2	5	36

Block-Words, Logistics, and Intrusion-Detection ship as static PDDL files in the authors’ archive. Grid ships only two instances, so we hand-expanded the set to eight by writing six new instances on top of the two existing layouts. The new instances vary only in their candidate goal sets. The grid topology, key positions, and starting position are kept. Campus and Kitchen ship as Python 2 generator scripts that synthesise instances at random. We ported these to Python 3 and wrote a driver that runs the generator and expands the resulting archives into the same per-hypothesis layout used by the other domains.

4.3 Hyperparameters

The only hyperparameter in the recognition pipeline is the Boltzmann temperature β , which the original paper does not specify. We use $\beta = 1.0$ for all headline results. As part of the extension analysis in Section 5 we sweep $\beta \in \{0.1, 0.5, 1.0, 2.0, 5.0, 10.0\}$ and report posterior probability mass on the true goal at each setting.

A second implicit choice concerns how observation sequences are sampled. The paper takes 10%, 30%, 50%, 70% or 100% of the actions of the hidden optimal plan, but does not say whether the sample is a prefix of the plan or a uniform random subset. We chose prefix sampling. For each trial, the first $\lceil p \cdot |\pi^*| \rceil$ actions of the optimal plan for the true goal are taken as the observation sequence O . This choice may account for some of the difference we see at low observation percentages.

One consequence of sampling observation prefixes from optimal plans is that the recognition quality Q saturates at 1.00 by construction. The compliant problem for the true goal always has a cost-zero solution, since the optimal plan for the true goal already contains O as its prefix. So the true goal’s cost difference is at most as large as any wrong goal’s, and the true goal is therefore at least tied for the highest posterior in every trial. As a result, Q is uninformative in our data and the substantive content of Claims 1 and 2 is carried by the spread metric instead. We discuss this in detail in Section 5.

4.4 Experimental setup and code

For each (domain, instance, true goal index, observation %) combination, the evaluation driver:

1. Locates the pre-computed optimal plan for the true goal (`OPT_*_hyp_K.soln`). If absent, it is computed once and cached.
2. Truncates the plan to the first $\lceil p \cdot |\pi^*| \rceil$ actions to form O .
3. For each of the K candidate goals, compiles two PDDL pairs ($G_k + O$ and $G_k + \overline{O}$), solves both with Fast Downward, and records the cost difference $\Delta(G_k, O)$.
4. Computes the posterior using the Boltzmann likelihood, identifies the goals tied at the maximum probability, and writes the trial’s outcome to a CSV.

Fast Downward is invoked with `-search "astar(lmcut())"`, yielding optimal-cost plans under the landmark-cut admissible heuristic. This configuration is the closest practical analogue to the paper’s HSP_f^* optimal planner.

Evaluation metrics follow the paper’s definitions. Q is the fraction of trials in which the true goal is in the tied top-probability set, and $|G^*|$ is the average size of that set. We additionally report strict accuracy (predicted index exactly equals true index) and posterior mass on the true goal $P(G^* | O)$, the latter being our β -sensitive metric.

All code is available at https://github.com/zaid-ismail031/COMS7044A_Reproducibility_Assignment.

Upstream PDDL corrections. Two domain files in the authors’ archive contain syntax errors that prevent Fast Downward’s translator from parsing them. We patched both, documented the fixes in the repository, and noted them here for transparency:

- `block-words/domain.pddl` contained the declaration `(holding ?x -block)`, missing a space between `-` and `block`. Fixed to `(holding ?x - block)`.
- `kitchen/domain.pddl` listed `cup`, `sugar`, and `bread` twice in the `:constants` block, and listed `toaster` as both an object and a useable. We removed the duplicates and kept `toaster` as a useable only (its only use site).

Both fixes are purely syntactical and do not alter the semantics of the domain.

4.5 Computational requirements

Experiments were run inside WSL2 (Ubuntu 24.04) on an Intel(R) Core(TM) i7-12700H machine with 64GB RAM. Fast Downward was compiled from source against `g++-11`.

Each recognition trial requires $2K$ Fast Downward invocations, where K is the number of candidate goals for the instance. The total number of trials per β value is 1885. Total wall-clock time for the $\beta = 1.0$ run was approximately 2 hours. The full β sweep across six values required approximately 15.7 hours. This is dominated by Fast Downward’s search time on the compiled problems. The pipeline is fully serial and does not use the GPU.

5 Results

We ran 1885 recognition trials per β value across the six benchmark domains, with trial counts per (domain, observation %) cell as summarised in Table 1. We report two ways of viewing the data. The first compares our values for Q and $|G^*|$ directly against Table 1 of [Ramírez and Geffner 2010]. The second is an additional measurement of $P(G^* | O)$ across the β sweep, which the original paper does not report.

Summary of findings. Two of the four claims from Section 2 are supported by our data without qualification. Claim 1 (monotonic non-increasing spread) holds in all six of our domains and in four of six in the paper. Claim 4 (the β extension) is supported by the sweep. The other two are partial. Claim 2 ($|G^*| \leq 2$ by 50% observation) holds in five of six domains in both the paper and our reproduction, with Block-Words as the shared failure mode. Claim 3 (spread converges to one at full observation) holds in four of six domains in both reproductions. Across all directly comparable cells the paper’s Q values reproduce exactly. The spread $|G^*|$ is systematically larger than the paper’s at low observation percentages, particularly on Block-Words and Intrusion-Detection, and converges to the paper’s values at full observation. The β sweep shows that for integer-cost domains the rank-based metrics Q and $|G^*|$ are mathematically invariant under β . Only the posterior probability mass on the true goal $P(G^* | O)$ varies with β .

5.1 Results reproducing original paper

Tables 2 and 4 show the cell-by-cell comparison of our recognition quality Q and spread $|G^*|$ against the values reported in the original paper’s Table 1 (HSP_f^{*} column, the optimal planner). The three subsections below discuss the evidence for each of the first three claims of Section 2 in turn.

5.1.1 Claim 1: Q is non-decreasing in observation percentage

Table 2 shows recognition quality Q in our reproduction alongside the values from the paper’s HSP_f^{*} column. In our reproduction, Q reaches 1.00 at every observation percentage in every domain. The paper’s column is also non-decreasing across every row, with the lowest entry at Easy IPC Grid at 10% observation ($Q = 0.75$) rising to 1.00 by 30% observation. Under Q alone, Claim 1 is satisfied trivially, because Q saturates at the ceiling in almost every cell.

Q is monotonically non-decreasing because our paper’s experimental setup guarantees the true goal is never ruled out. We always sample from an optimal plan, whereas the paper sometimes does not. The paper’s Experimental Results section says explicitly:

"The observation sequences are generated by sampling hidden optimal plans for a hidden goal in the first four domains, and by sampling hidden suboptimal plans for a hidden goal, in the others."

So for Block-Words, Logistics, Grid, and Intrusion-Detection, the paper does what we do. It samples from an optimal plan. For Campus and Kitchen, the paper samples from a suboptimal plan that the generator simulated stochastically. We don't. We use the precomputed optimal plan from Fast Downward in every domain. This choice was made to ensure a uniform evaluation pipeline.

The more informative view of the same claim is the spread metric, which captures the recogniser's decisiveness. A non-decreasing Q paired with a falling spread tells us that the recogniser is keeping the true goal in its top set *and* narrowing that set as it sees more of the agent's actions. Reading Table 4 row by row, our spread is monotonically non-increasing in all six domains. The paper's spread is monotonic in four of six domains. Kitchen and Logistics in the paper show small non-monotonic dips at 50% observation (Kitchen rises from 1.21 to 1.33, Logistics from 1.07 to 1.20), likely artefacts of the small sample sizes in those cells.

Claim 1 is supported in every domain by both the paper and our reproduction. Under Q the support is trivial. Under spread, our reproduction is in fact stricter than the original paper, since our spread is monotonic in every domain.

Table 2: **Recognition quality Q (lenient correctness):** ours vs. paper. A trial counts as correct if the true goal is among the goals tied for the highest posterior probability, regardless of how many other goals are tied with it. This is the metric reported in Table 1 of the original paper. Contrast with Table 3, which requires a single, unique prediction to match the true goal.

Domain	Observation %				
	10	30	50	70	100
<i>Paper</i>					
Block-Words ($ \mathcal{G} =20$)	1.00	1.00	1.00	1.00	1.00
Campus ($ \mathcal{G} =2$)	0.93	1.00	1.00	1.00	1.00
Easy IPC Grid ($ \mathcal{G} =7.5$)	0.75	1.00	1.00	1.00	1.00
Intrusion Detection ($ \mathcal{G} =15$)	1.00	1.00	1.00	1.00	1.00
Kitchen ($ \mathcal{G} =3$)	0.88	0.93	1.00	1.00	1.00
Logistics ($ \mathcal{G} =10$)	0.90	1.00	1.00	1.00	1.00
<i>Ours</i>					
Block-Words ($ \mathcal{G} =20$)	1.00	1.00	1.00	1.00	1.00
Campus ($ \mathcal{G} =2$)	1.00	1.00	1.00	1.00	1.00
Grid ($ \mathcal{G} =2.6$, see note)	1.00	1.00	1.00	1.00	1.00
Intrusion Detection ($ \mathcal{G} =15$)	1.00	1.00	1.00	1.00	1.00
Kitchen ($ \mathcal{G} =3$)	1.00	1.00	1.00	1.00	1.00
Logistics ($ \mathcal{G} =7.2$)	1.00	1.00	1.00	1.00	1.00

5.1.2 Claim 2: $|G^*| \leq 2$ by 50% observation

Reading the 50% column of Table 4, the spread is at most 2 in five of the six domains for both the paper and our reproduction. The exception in both is Block-Words. The paper reports a Block-Words spread of 2.23. Our reproduction reports 4.75.

The fact that Block-Words is the common failure mode in both reproductions is consistent with the structure of the domain. Block-Words has twenty candidate goals per instance, and many of those goals share long stretches of their optimal plans (the same blocks need to be unstacked and restacked regardless of which word is being spelled). Observations from the first half of an optimal plan are therefore not enough to disambiguate among related word goals.

We additionally report strict accuracy in Table 3, which the paper does not. Strict accuracy is a tougher measure than Q . A trial counts as correct only when the recogniser commits to a single goal that is exactly the true goal. Trials in which the true goal is tied with one or more other goals count as wrong, even though they count as right under Q . The two tables therefore measure two different things on the same data set. Table 2 asks whether the true goal was *not ruled out*, and Table 3 asks whether the true goal was *uniquely identified*. Under the stricter measure, accuracy at 50% observation is at least 0.9 in Campus, Kitchen and Logistics. Intrusion-Detection comes in just below at 0.83. Block-Words at 0.35 and Grid at 0.71 fall well below.

Claim 2 is supported in five of six domains by both the paper and our reproduction. The shared failure mode is Block-Words. Our spread on Block-Words at 50% observation is roughly twice the paper's. We attribute this to two

Table 3: **Strict accuracy (unique-prediction correctness)** at $\beta = 1.0$, ours only. A trial counts as correct only if the recogniser’s single most-likely prediction exactly equals the true goal. Tied top sets count as wrong. Always at most Q from Table 2 on the same data. The paper does not report this metric.

Domain	Observation %				
	10	30	50	70	100
Block-Words	0.08	0.23	0.35	0.60	0.80
Campus	1.00	1.00	1.00	1.00	1.00
Grid	0.67	0.67	0.71	0.71	0.81
Intrusion-Detection	0.27	0.70	0.83	1.00	1.00
Kitchen	0.33	1.00	1.00	1.00	1.00
Logistics	0.58	0.89	0.89	0.89	1.00

implementation choices the paper does not specify: the value of the Boltzmann temperature β and the observation-sampling strategy. Both are discussed in Section 4 and revisited in the Discussion.

5.1.3 Claim 3: $|G^*|$ converges to one at 100% observation

Table 4 reports spread at every cell. At 100% observation, our spread is 1.00 for Campus, Intrusion-Detection, Kitchen, and Logistics. It is 1.40 for Block-Words and 1.52 for Grid. The paper reports 1.00 for Campus, Easy IPC Grid, Intrusion-Detection, and Logistics, 1.13 for Block-Words and 1.47 for Kitchen.

Claim 3 is supported in four of the six domains in both the paper and our reproduction. Block-Words fails to converge in both, suggesting that some block configurations admit multiple equal-cost spelling solutions even with the full plan observed. Kitchen fails to converge in the paper but does in our reproduction. Grid fails in our reproduction but does in the paper. Our Grid result reflects a different instance set, not a different method.

Table 4: **Spread $|G^*|$ (denoted S in the original paper):** ours vs. paper. $|G^*|$ is the average size of the tied top-probability set across trials. Lower is better. A value of 1.00 means every trial picked a single, unique winner.

Domain	Observation %				
	10	30	50	70	100
<i>Paper</i>					
Block-Words	6.00	3.25	2.23	1.27	1.13
Campus	1.33	1.00	1.00	1.00	1.00
Easy IPC Grid	1.38	1.00	1.00	1.00	1.00
Intrusion Detection	1.80	1.13	1.00	1.00	1.00
Kitchen	1.25	1.21	1.33	1.20	1.47
Logistics	2.30	1.07	1.20	1.00	1.00
<i>Ours</i>					
Block-Words	12.82	6.62	4.75	2.35	1.40
Campus	1.04	1.00	1.00	1.00	1.00
Grid	2.00	1.95	1.81	1.71	1.52
Intrusion Detection	4.07	1.83	1.27	1.00	1.00
Kitchen	2.00	1.33	1.00	1.00	1.00
Logistics	2.17	1.28	1.28	1.22	1.00

5.2 Results beyond original paper

The original paper writes the likelihood as $P(O \mid G) \propto \exp(-\beta \cdot \Delta(G, O))$ without specifying a value for the temperature β . We swept $\beta \in \{0.1, 0.5, 1.0, 2.0, 5.0, 10.0\}$ and measured the resulting posterior probability mass $P(G^* \mid O)$ on the true goal. The headline observation is that all of our domains have integer action costs, which means the cost differences Δ are integer-valued. The Boltzmann transform is monotonic in $-\beta \cdot \Delta$, so the rank of each candidate is preserved across any positive β . This makes the rank-based metrics Q and $|G^*|$ mathematically invariant under β . Their values in the comparison tables above would be unchanged at any positive β .

5.2.1 Claim 4: β affects $P(G^* \mid O)$ but not Q or $|G^*|$

What does change with β is the actual probability mass on the true goal, $P(G^* \mid O)$. Table 5 shows the values across the sweep. A low β produces a nearly uniform posterior, where the true goal receives close to $1/|\mathcal{G}|$ of the mass. A high β produces a sharp posterior. Mass concentrates on the goals with the lowest cost difference, capped at $1/|G^*|$ since β cannot break ties between candidates with identical Δ .

Claim 4 is supported by the data. The Q and $|G^*|$ values in our β sweep remain identical across the six β settings, while $P(G^* \mid O)$ varies across the sweep, in some cells by an order of magnitude.

Table 5: Posterior probability mass on the true goal, $P(G^* \mid O)$, as a function of β . Computed by averaging the posterior probability assigned to the true goal across all trials in each (domain, observation %) cell. β affects this metric even though Q and $|G^*|$ are β -invariant for integer-cost domains.

Domain	Obs %	$\beta=0.1$	$\beta=0.5$	$\beta=1.0$	$\beta=2.0$	$\beta=5.0$	$\beta=10.0$
block-words	10	0.05	0.06	0.07	0.08	0.08	0.08
	30	0.06	0.10	0.14	0.20	0.22	0.22
	50	0.07	0.14	0.22	0.29	0.31	0.31
	70	0.08	0.27	0.44	0.56	0.61	0.61
	100	0.10	0.52	0.78	0.85	0.85	0.85
campus	10	0.54	0.67	0.79	0.91	0.98	0.98
	30	0.56	0.78	0.91	0.99	1.00	1.00
	50	0.61	0.90	0.98	1.00	1.00	1.00
	70	0.65	0.96	1.00	1.00	1.00	1.00
	100	0.70	0.98	1.00	1.00	1.00	1.00
grid	10	0.39	0.44	0.48	0.54	0.59	0.59
	30	0.41	0.50	0.56	0.59	0.60	0.60
	50	0.43	0.56	0.61	0.65	0.66	0.66
	70	0.46	0.64	0.68	0.69	0.69	0.69
	100	0.51	0.72	0.77	0.78	0.79	0.79
intrusion-detection	10	0.07	0.12	0.17	0.24	0.29	0.30
	30	0.10	0.27	0.48	0.64	0.70	0.71
	50	0.12	0.48	0.76	0.86	0.87	0.87
	70	0.15	0.69	0.90	0.97	1.00	1.00
	100	0.20	0.91	0.99	1.00	1.00	1.00
kitchen	10	0.34	0.39	0.44	0.53	0.61	0.61
	30	0.38	0.56	0.66	0.75	0.83	0.83
	50	0.43	0.69	0.82	0.94	1.00	1.00
	70	0.48	0.75	0.88	0.96	1.00	1.00
	100	0.53	0.83	0.95	0.99	1.00	1.00
logistics	10	0.15	0.22	0.31	0.46	0.60	0.61
	30	0.19	0.44	0.67	0.82	0.86	0.86
	50	0.21	0.57	0.80	0.86	0.86	0.86
	70	0.26	0.75	0.87	0.89	0.89	0.89
	100	0.34	0.93	0.99	1.00	1.00	1.00

6 Discussion

Our results support the core claim of the original paper. The method finds the true goal in every domain at every observation percentage we tested, in the sense that the true goal is always in the set of most likely candidates under the posterior. The shape of the recognition quality curve matches the paper. The decision to use Fast Downward with the landmark-cut heuristic reproduces the paper’s reliance on an optimal admissible heuristic.

The main quantitative gap is in the spread metric at low observation percentages. Our $|G^*|$ at 10% observation is roughly twice the paper’s on Block-Words and Intrusion-Detection. Two factors may explain this. We sample observations as a prefix of the optimal plan, while the paper does not state its sampling method. A prefix gives less information about the agent’s goal than a uniform sample drawn across the whole plan, especially at the start. The paper may also use a higher β in places, which would reduce spread by breaking near-ties. Without more detail in the original paper we cannot pin down which factor is dominant.

The Grid result is bounded by the size of the upstream benchmark. Only two grid instances ship with the authors’ archive, and they share a small candidate goal set. We hand-expanded the set with six additional instances built on the same two layouts, which gave us $|\mathcal{G}|$ of about 2.6 against the paper’s 7.5. This makes our Grid task strictly easier than the paper’s, so our cell values are not directly comparable. A clean comparison would require generating new grid layouts with larger candidate sets, which is outside the scope of this reproduction.

The β extension turned out to be a clean negative result that strengthens the paper’s main claim. For integer-cost domains, no value of β affects the ranking of candidates. The paper’s headline Q and $|G^*|$ tables are therefore unaffected

by the unspecified β . The posterior probability mass on the true goal is not, and the paper’s silence on β matters for any downstream application that relies on those probabilities directly.

6.1 What was easy

Definition 2 and Proposition 3 of the paper are short and clear. The compilation took about a hundred lines of Python and worked first try once the parameter substitution edge case was handled. Fast Downward solved every compiled instance in sub-second time on the smaller domains. The Bayesian scoring layer is twenty lines of code. The optimal plans that ship with Grid and Intrusion-Detection let us start the recognition loop without first running the planner over every instance in those domains.

6.2 What was difficult

The paper does not specify β . We spent some time chasing what we believed was a bug in the scoring code before realising that β has no effect on the rank-based metrics we were measuring. We wrote the β sweep as an extension once this was clear.

Two of the shipped PDDL files have syntax errors that Fast Downward rejects. The Block-Words domain was missing a space between a hyphen and a type name. The Kitchen domain had duplicate constants. Both are purely syntactical fixes, but they block the pipeline until found.

Campus and Kitchen ship as Python 2 generator scripts. Porting them to Python 3 and wrapping them in a driver that produces the same layout as the other domains took longer than expected, partly because the generators rely on a tar pipeline that expects bzip2 on the host.

Four of the six domains do not ship with optimal solutions. We had to run Fast Downward over every problem in those domains as a one-time precomputation step before recognition could begin.

References

- [Cohen *et al.* 1981] Philip R. Cohen, C. Raymond Perrault, and James F. Allen. Beyond question answering. In *Strategies for Natural Language Processing*. Lawrence Erlbaum Associates, 1981.
- [Elsevier nd] Elsevier. *Goal Recognition*. <https://www.sciencedirect.com/topics/computer-science/goal-recognition>, n.d. ScienceDirect Topics. Accessed: 2026-05-19.
- [Geib and Goldman 2002] Christopher W. Geib and Robert Goldman. Requirements for plan recognition in network security systems. In *Proceedings of the International Symposium on Recent Advances in Intrusion Detection*, 2002.
- [Geib and Goldman 2009] Christopher W. Geib and Robert P. Goldman. A probabilistic plan recognition algorithm based on plan tree grammars. *Artificial Intelligence*, 173(11):1101–1132, 2009.
- [Helmert 2006] Malte Helmert. The fast downward planning system. volume 26, pages 191–246, 2006.
- [Huber *et al.* 1994] Marcus J. Huber, Edmund H. Durfee, and Michael P. Wellman. The automated mapping of plans for plan recognition. In *Proceedings of the Tenth Conference on Uncertainty in Artificial Intelligence (UAI-94)*, pages 344–351, 1994.
- [Kautz and Allen 1986] Henry Kautz and James F. Allen. Generalized plan recognition. In *Proceedings of the Fifth National Conference on Artificial Intelligence (AAAI-86)*, pages 32–37, 1986.
- [Pentney *et al.* 2006] William Pentney, Adrian Popescu, Shiao-kai Wang, Henry Kautz, and Matthai Philipose. Sensor-based understanding of daily life via large-scale use of common sense. In *Proceedings of the Twenty-First National Conference on Artificial Intelligence (AAAI-06)*, 2006.
- [Ramírez and Geffner 2009] Miquel Ramírez and Hector Geffner. Plan recognition as planning. In *Proceedings of the 21st International Joint Conference on Artificial Intelligence (IJCAI-09)*, pages 1778–1783, 2009.
- [Ramírez and Geffner 2010] Miquel Ramírez and Hector Geffner. Probabilistic plan recognition using off-the-shelf classical planners. In *Proceedings of the Twenty-Fourth AAAI Conference on Artificial Intelligence (AAAI-10)*, pages 1121–1126, 2010.